

Vocabulaire commun — SAE-Recommender

Projet : SAÉ BUT2 S3 2026-2027 **Thème :** Langages de programmation **Date :** 16 septembre 2026 - 18/09/2026

1. Objectif

Ce document définit le vocabulaire commun utilisé par tous les membres du groupe. Il garantit que le client Java, le serveur C et la base de données parlent le même langage.

2. Nature des cartes

Une carte représente un **langage de programmation**.

Justification :

- Cohérent avec notre formation (BUT Informatique)
- Objet concret et vérifiable
- Facilité de description avec des critères objectifs
- Permet des battles pertinentes
- Chaque langage possède un logo officiel

Exemples : Python, Java, C, C++, Rust, JavaScript, TypeScript, Go, Ruby, Kotlin, Swift, Haskell, PHP, Scala, Elixir, Lua, R, SQL, Bash

3. Champs d'une carte

Champ	Type	Obligatoire	Description	Exemple
id	String	✓	Identifiant unique	c_001
title	String	✓	Nom du langage	Python
paradigm	Enum	✓	Paradigme principal	MULTI
description	String	✓	Description	Langage interprété
typing	Enum	✓	Type de typage	DYNAMIC
difficulty	Enum	✓	Difficulté	BEGINNER
yearCreated	int	×	Année de création	1991
imageUrl	String	×	URL du logo	https://...
creator	String	✓	Créateur	u_001
owner	String	✓	Propriétaire courant	u_001
status	Enum	✓	Statut	ACTIVE

Champ	Type	Obligatoire	Description	Exemple
value	int	✓	Nombre de recos (VA)	12
isAuthenticated	boolean	✓	Authentifiée ?	false
authenticatedBy	String	×	Qui a authentifié	u_003
authenticatedAt	String	×	Quand	2026-09-25T11:30:00Z
winCount	int	✓	Victoires en battle	3
isLegitimate	boolean	✓	Propriétaire légitime ?	false
stakeVA	int	✓	VA engagée	0
createdAt	String	✓	Date de création	2026-09-25T11:30:00Z

4. Valeurs des énumérations

paradigm

Valeur	Signification	Exemples
OBJECT	Orienté objet	Java, C++
FUNCTIONAL	Fonctionnel	Haskell, Elixir
PROCEDURAL	Procédural	C, Pascal
LOGIC	Logique	Prolog
SCRIPTING	Script	Bash
MULTI	Multi-paradigme	Python, Rust

typing

Valeur	Signification	Exemples
STATIC	Typage statique	Java, C
DYNAMIC	Typage dynamique	Python, JS
STRONG	Typage fort	Python, Java
WEAK	Typage faible	C, JS

difficulty

Valeur	Signification	Exemples
BEGINNER	Débutant	Python
INTERMEDIATE	Intermédiaire	Java, JS

Valeur	Signification	Exemples
ADVANCED	Avancé	C, C++
EXPERT	Expert	Haskell

status

Valeur	Signification
ACTIVE	Carte active
INACTIVE	Carte inactive

5. Format des identifiants

Format : **<préfixe>_<numéro>**

Type	Préfixe	Format	Exemples
Carte	c	c_XXX	c_001, c_042
Utilisateur	u	u_XXX	u_001, u_002
Battle	b	b_XXX	b_001, b_002
Recommandation	r	r_XXX	r_001
Vote	v	v_XXX	v_001
Légitimation	l	l_XXX	l_001

6. Format des timestamps

Format : **ISO 8601 UTC**

Structure : **YYYY-MM-DDTHH:MM:SSZ**

Exemple : **2026-09-25T11:30:00Z**

7. Format des messages

- Format : **JSON**
- Encodage : **UTF-8**
- Délimiteur : **\n**
- Transport : **Socket TCP**

Requête :

```
{"action": "LOGIN", "payload": {"username": "usman"}}
```

Réponse succès :

json

```
{"status":"OK","action":"LOGIN_SUCCESS","data":{"userId":"u_001"}}
```

Réponse erreur :

json

```
{"status":"ERROR","code":"USERNAME_TAKEN","message":"Pseudo deja pris"}
```

8. Liste des actions

#	Action	Description
-:	-----	-----
1	`LOGIN`	Connexion
2	`GET_CONTEXT`	Récupérer le contexte
3	`CREATE_CARD`	Créer une carte
4	`RECOMMEND`	Recommander
5	`REPUDIATE`	Répudier
6	`START_BATTLE`	Lancer battle
7	`VOTE_BATTLE`	Voter
8	`REQUEST_LEGITIMACY`	Demander légitimation
9	`CLAIM_CARD`	Revendiquer
10	`AUTHENTICATE_CARD`	Authentifier
11	`DISCONNECT`	Déconnexion

9. Notifications push

#	Action	Quand
-:	-----	-----
1	`USER_CONNECTED`	Connexion
2	`USER_DISCONNECTED`	Déconnexion
3	`NEW_CARD`	Nouvelle carte
4	`CARD_UPDATED`	VA modifiée
5	`NEW_BATTLE`	Battle lancée
6	`BATTLE_UPDATE`	Vote enregistré
7	`BATTLE_RESULT`	Battle terminée
8	`LEGITIMACY_GRANTED`	Légitimation
9	`NEW_NOTIFICATION`	Notification

10. Codes d'erreur

#	Code	Signification
-:	-----	-----
1	`INVALID_DATA`	Champs invalides
2	`USERNAME_TAKEN`	Pseudo pris
3	`CARD_NOT_FOUND`	Carte inexistante
4	`CARD_NOT_ACTIVE`	Carte inactive

5	`DUPLICATE_CARD`	Carte existante
6	`ALREADY_RECOMMENDED`	Déjà recommandée
7	`NOT_RECOMMENDED`	Rien à répudier
8	`BATTLE_NOT_FOUND`	Battle inexistante
9	`BATTLE_ALREADY_ACTIVE`	Carte en battle
10	`ALREADY_VOTED`	Déjà voté
11	`NOT_LEGITIMATE`	Non légitime
12	`THRESHOLD_NOT_REACHED`	Seuil non atteint
13	`CARDS_NOT_COMPATIBLE`	Paradigmes différents
14	`VA_TOO_LOW`	VA < 5
15	`ALREADY_AUTHENTICATED`	Déjà authentifiée
16	`SERVER_BUSY`	Serveur plein
17	`INTERNAL_ERROR`	Erreur interne

11. Points techniques

Point	Valeur
-----	-----
Timeout connexion	5 secondes
Timeout lecture	10 secondes
Max clients	10
Encodage	UTF-8
Délimiteur	`\n`

12. Structure d'un bloc blockchain

Champ	Type	Description
-----	-----	-----
`id`	`int`	Identifiant unique
`timestamp`	`long`	Date de création (epoch)
`data`	`String`	Données de l'action (JSON)
`prevHash`	`String`	Hash du bloc précédent
`nonce`	`int`	Preuve de travail
`hash`	`String`	Hash courant

****Algorithme :*** SHA-256

****Difficulté :*** 3 ou 4

****Premier bloc :*** `genesis`

13. Récapitulatif

Élément	Valeur
-----	-----
Thème	Langages de programmation
Format des IDs	`c_001`, `u_001`, `b_001`
Format des timestamps	ISO 8601 UTC
Format des messages	JSON + `\n`
Nombre d'actions	11

Nombre de notifications	9	
Nombre de codes d'erreur	17	
Timeout	5s connexion, 10s lecture	
Max clients	10	

****Fin du document.****